# DSBDA Lab Practicals

Complete Code Explanation with Comments

| # | Practical | Topic |
|---|---|---|
| 1 | Data Wrangling 1 | IRIS – Normalization, Label & One-Hot Encoding |
| 2 | Data Wrangling 2 | Students – Missing Values, Outliers, Log Transform |
| 3 | Descriptive Statistics | Mall Customers & IRIS – Mean, Median, Mode, Std |
| 4 | Data Analytics 1 | Linear Regression – Simple & California Housing |
| 5 | Data Analytics 2 | Logistic Regression & Confusion Matrix Metrics |
| 6 | Data Analytics 3 | Naive Bayes Classifier |
| 7 | Text Analytics | NLTK – Tokenize, Stem, Lemmatize, POS, Sentiment |
| 8 | Data Visualization 1 | Seaborn Plots – Titanic Dataset |
| 9 | Data Visualization 2 | Boxplots – Titanic & Tips Datasets |
| 10 | Data Visualization 3 | Histograms & Boxplots – Generic Dataset |

# Practical 1 – Data Wrangling 1

Dataset: IRIS.csv | Goal: Normalize features and encode the species label using Label Encoding and One-Hot Encoding.

---

## 1. Load & Inspect Data

```
# Import pandas for DataFrames and numpy for numerical operations
import pandas as pd import numpy as np
# Load IRIS dataset — 150 rows, 5 columns (4 features + species)
df = pd.read_csv("IRIS.csv") df.head() # Preview first 5 rows df.dtypes # Check data types of each column
```

## 2. Min-Max Normalization

Normalization scales all feature values to the range [0, 1]. This prevents features with large ranges from dominating the model.

Formula: x_scaled = (x - x_min) / (x_max - x_min)

```
# MinMaxScaler from sklearn performs feature normalization
from sklearn import preprocessing min_max_scaler = preprocessing.MinMaxScaler()
# Select only the 4 numeric feature columns (exclude 'species')
x = df.iloc[:, :4]
# fit_transform: learns min/max from data, then scales it
x_scaled = min_max_scaler.fit_transform(x) df_normalized = pd.DataFrame(x_scaled) # Convert back to DataFrame
```

## 3. Label Encoding

Converts string categories to integers: Iris-setosa→0, Iris-versicolor→1, Iris-virginica→2. Useful when the model needs numeric input.

```
# Show unique species names before encoding
df['species'].unique()
# LabelEncoder assigns a unique integer to each category
label_encoder = preprocessing.LabelEncoder() df['species'] = label_encoder.fit_transform(df['species']) df['species'].unique() # Now shows [0, 1, 2]
```

## 4. One-Hot Encoding (Manual via sklearn)

Creates a binary column for each category. Avoids the false numeric order implied by Label Encoding (0 < 1 < 2).

```
# Drop 'species' to get pure feature columns
features_df = df.drop(columns=['species'])
# OneHotEncoder creates 3 binary columns: one per class
enc = preprocessing.OneHotEncoder() enc_df = pd.DataFrame(enc.fit_transform(df[['species']]).toarray())
# Join encoded columns with feature columns
df_encode = features_df.join(enc_df)
# Rename binary columns with meaningful names
df_encode.rename(columns={0:'Iris-Setosa', 1:'Iris-Versicolor', 2:'Iris-virginica'}, inplace=True)
```

## 5. One-Hot Encoding (Shortcut via pd.get_dummies)

pandas get_dummies() is a quicker way to one-hot encode. drop_first=True removes one column to avoid multicollinearity.

```
# pd.get_dummies is simpler — no manual fit/transform needed
one_hot_df = pd.get_dummies(df, prefix="Species", columns=["species"], drop_first=True)
```

***Note:*** *drop_first=True drops the first dummy column to avoid dummy variable trap (perfect multicollinearity).*

# Practical 2 – Data Wrangling 2

Dataset: StudentsPerformanceTest1.csv & demo1.csv | Goal: Handle missing values, detect outliers, and apply log transformation.

---

## 1. Load & Check Null Values

```
# Read student performance data
df = pd.read_csv("StudentsPerformanceTest1(1).csv")
# isnull() returns True where data is missing
df.isnull()
# Filter rows where math score is missing
series = pd.isnull(df["math score"]) df[series]
# Filter rows where math score is present (not null)
series1 = pd.notnull(df["math score"]) df[series1]
```

## 2. Label Encoding on Gender Column

```
# Encode gender: female->0, male->1 (or vice versa alphabetically)
from sklearn.preprocessing import LabelEncoder le = LabelEncoder() df["gender"] =
le.fit_transform(df["gender"])
```

## 3. Handling Missing Values with Custom na_values

```
# Tell pandas to treat 'Na' and 'na' strings as NaN
missing_values = ["Na", "na"] df = pd.read_csv("StudentsPerformanceTest1(1).csv",
na_values=missing_values)
```

## 4. Filling Missing Values – Different Strategies

Different columns use different fill strategies depending on data distribution.

```
# Fill missing math scores with column MEAN (good for normally distributed data)
m_v = df["math score"].mean() df["math score"].fillna(value=m_v, inplace=True)
# Fill reading score with MEDIAN (better when data is skewed)
me_v = df["reading score"].median() df["reading score"].fillna(value=me_v, inplace=True)
# Fill writing score with STD (less common — used as a placeholder)
std_v = df["writing score"].std() df["writing score"].fillna(value=std_v, inplace=True)
# Fill placement score with MIN value
min_v = df["Placement Score"].min() df["Placement Score"].fillna(value=min_v, inplace=True)
# Fill placement offer count with MAX value
max_v = df["placement offer count"].max() df["placement offer count"].fillna(value=max_v,
inplace=True)
```

## 5. Outlier Detection – Boxplot

```
# Boxplot shows IQR; points beyond whiskers are outliers
col = ["math score", "reading score", "writing score", "placement score"] df.boxplot(col)
```

## 6. Scatter Plot & Log Transformation

```
# Scatter plot reveals relationship between placement score and offer count
fig, ax = plt.subplots(figsize=(18, 10)) ax.scatter(df["placement score"], df["placement offer
count"]) plt.show()
```

```
# Log transform compresses skewed data, making distribution more normal
df["math score"].plot(kind="hist") # Original histogram df["log_math"] = np.log10(df["math score"]) df["log_math"].plot(kind="hist") # After log transform
```

# Practical 3 – Descriptive Statistics

Datasets: Mall_Customers.csv & IRIS.csv | Goal: Compute and interpret central tendency and spread statistics.

---

## 1. Central Tendency – Mean, Median, Mode

```
# Mean: average of all values
df.mean() # Mean of all numeric columns df.loc[:, 'Age'].mean() # Mean of Age column only
df.mean(axis=1)[0:4] # Row-wise mean for first 4 rows
# Median: middle value — unaffected by outliers
df.median() df.loc[:, 'CustomerID'].median()
# Mode: most frequently occurring value
df.mode() df.loc[:, 'Age'].mode()
```

## 2. Min, Max & Standard Deviation

```
# Max/Min: extreme values; skipna=False includes NaN in computation
df.max() df.loc[:, 'Age'].max(skipna=False) df.min() df.loc[:, 'Age'].min(skipna=False)
# Std deviation: how spread out values are from the mean
df.std() df.loc[:, 'Age'].std() df.std(axis=1)[0:4] # Row-wise std deviation
```

## 3. GroupBy – Mean by Category

```
# GroupBy 'Genre' and compute average age per gender group
df.groupby(['Genre'])['Age'].mean()
```

## 4. One-Hot Encoding on Mall Dataset

```
# Rename column first (typo fix), then apply OneHotEncoder on Genre
df_u = df.rename(columns={'Annual Income k$)': 'Income'}, inplace=False) enc =
preprocessing.OneHotEncoder() enc_df =
pd.DataFrame(enc.fit_transform(df[['Genre']]).toarray()) df_encode = df_u.join(enc_df)
```

## 5. Describe by IRIS Species

describe() gives count, mean, std, min, 25%, 50%, 75%, max — a full statistical summary per species.

```
# Filter rows for each species and describe them separately
irisSet = (iris['species'] == 'Iris-setosa') print(iris[irisSet].describe())
# Same for versicolor and virginica
irisVer = (iris['species'] == 'Iris-versicolor') print(iris[irisVer].describe()) irisVir =
(iris['species'] == 'Iris-virginica') print(iris[irisVir].describe())
```

# Practical 4 – Data Analytics 1: Linear Regression

Two parts: (A) Manual linear regression on small data using numpy. (B) Multi-feature regression on California Housing dataset using sklearn.

---

## Part A – Simple Linear Regression (numpy)

```
# Sample x and y data points (e.g., marks in two subjects)
x = np.array([95, 85, 80, 70, 60]) y = np.array([85, 95, 70, 65, 70])
# polyfit finds best-fit line y = mx + b using least squares method # degree=1 means linear (straight line)
model = np.polyfit(x, y, 1) # Returns [slope, intercept]
# poly1d wraps the model into a callable function
predict = np.poly1d(model) predict(65) # Predict y when x = 65
# Predict y for all known x values
y_pred = predict(x)
# R2 score: 1.0 = perfect fit, 0 = model explains nothing
from sklearn.metrics import r2_score r2_score(y, y_pred)
# Plot regression line in red
y_line = model[1] + model[0] * x # y = intercept + slope*x plt.plot(x, y_line, c='r') # Regression line plt.scatter(x, y_pred) # Predicted points plt.scatter(x, y, c='r') # Actual points
```

## Part B – Multi-Feature Linear Regression (sklearn)

```
# fetch_california_housing: ~20,000 rows, 8 features, target=house price
from sklearn.datasets import fetch_california_housing housing = fetch_california_housing() data = pd.DataFrame(housing.data) data.columns = housing.feature_names # MedInc, HouseAge, AveRooms, etc. data['PRICE'] = housing.target # Median house value (in $100k)
# Check for missing values – California dataset has none
data.isnull().sum()
# Separate features (X) and target (y)
x = data.drop(['PRICE'], axis=1) # 8 feature columns y = data['PRICE'] # Target column
# 80% train, 20% test; random_state=0 ensures reproducibility
from sklearn.model_selection import train_test_split xtrain, xtest, ytrain, ytest = train_test_split(x, y, test_size=0.2, random_state=0)
# LinearRegression fits y = b0 + b1*x1 + b2*x2 + ... + b8*x8
from sklearn.linear_model import LinearRegression lm = LinearRegression() lm.fit(xtrain, ytrain) # Learn coefficients from training data
# Predict on both train and test sets
ytrain_pred = lm.predict(xtrain) # In-sample predictions ytest_pred = lm.predict(xtest) # Out-of-sample predictions (more important)
# Create DataFrames of actual vs predicted for comparison # NOTE: second df assignment overwrites first – use df_train/df_test instead
df = pd.DataFrame(ytrain_pred, ytrain) df = pd.DataFrame(ytest_pred, ytest) # Overwrites df_train!
# These metrics should be called to evaluate model quality
from sklearn.metrics import mean_squared_error, r2_score # r2_score(ytest, ytest_pred) # mean_squared_error(ytest, ytest_pred)
```

***Note:*** *A higher R2 (closer to 1) and lower MSE indicate a better model. Always compare train vs test metrics to check for overfitting.*

# Practical 5 – Data Analytics 2: Logistic Regression & Confusion Matrix

Dataset: data5.csv | Goal: Binary classification using Logistic Regression; evaluate with confusion matrix metrics.

---

## 1. Load & Prepare Data

```
# Load dataset — assumed to be a binary classification problem (e.g., Purchase: Yes/No)
df = pd.read_csv('data5.csv')
# Drop 'Genre' (gender) column — not used as a feature here
df1 = df.drop("Genre", axis=1)
# Last column = target (y), all others = features (X)
y = df1.iloc[:, -1].values # Target variable X = df1.iloc[:, :-1].values # Feature matrix
# 75% train, 25% test; random_state=2 for reproducibility
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=2)
```

## 2. Train Logistic Regression Model

Logistic Regression predicts probability using the sigmoid function: P(y=1) = 1 / (1 + e^(-z)). Output > 0.5 is classified as class 1.

```
# Fit logistic regression — finds decision boundary between classes
LR = LogisticRegression() LR.fit(X_train, y_train) y_pred = LR.predict(X_test)
```

## 3. Confusion Matrix & Derived Metrics

The confusion matrix is a 2x2 table: TN (True Negative), FP (False Positive), FN (False Negative), TP (True Positive).

```
# ravel() flattens the 2x2 matrix into 4 values
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
```

| Metric | Formula | Meaning |
|---|---|---|
| Accuracy | (TP+TN)/(TP+TN+FP+FN) | Overall correct predictions |
| Precision | TP/(TP+FP) | Of predicted positives, how many are correct |
| Recall | TP/(TP+FN) | Of actual positives, how many were caught |
| F1 Score | 2*(P*R)/(P+R) | Harmonic mean of Precision and Recall |
| Specificity | TN/(TN+FP) | Of actual negatives, how many were correctly identified |

```
# Compute metrics manually from confusion matrix values
accuracy = (tn + tp) * 100 / (tp + tn + fp + fn) precision = tp / (tp + fp) recall = tp / (tp + fn) f1_score = (2 * precision * recall) / (precision + recall) specificity = tn / (tn + fp) print(f'Accuracy: {accuracy:.2f}%')
```

## 4. User Input for Prediction

```
# Accept feature values from user and predict class
input_features = [] for i in range(X_train.shape[1]): # Loop once per feature val = float(input(f'Feature {i+1}: ')) input_features.append(val) input_array = np.array(input_features).reshape(1, -1) # Shape: (1, n_features) prediction = LR.predict(input_array)[0] print(f'Predicted Output: {prediction}')
```

# Practical 6 – Data Analytics 3: Naive Bayes Classifier

Dataset: data6.csv (likely Social_Network_Ads — Age, Salary, Purchased) | Goal: Classify using Gaussian Naive Bayes.

---

## 1. Load Data & Split

```
# Load CSV — assumed to have features + 1 target column at the end
dataset = pd.read_csv('data6.csv') X = dataset.iloc[:, :-1].values # All columns except last y = dataset.iloc[:, -1].values # Last column = target
# 75/25 split, same random state as Practical 5 for comparison
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=2)
```

## 2. Gaussian Naive Bayes

Naive Bayes assumes each feature is independent and normally distributed within each class. It uses Bayes' theorem: P(class|features) proportional to P(features|class) * P(class).

```
# GaussianNB assumes continuous features follow a Gaussian (normal) distribution
from sklearn.naive_bayes import GaussianNB classifier = GaussianNB() classifier.fit(X_train, y_train) # Learn mean & variance per class per feature y_pred = classifier.predict(X_test)
```

## 3. Evaluate Model

```
# average='micro' sums TP, FP, FN across all classes before computing
accuracy = accuracy_score(y_test, y_pred) precision = precision_score(y_test, y_pred, average='micro') recall = recall_score(y_test, y_pred, average='micro')
# Confusion matrix — rows=actual, cols=predicted
cm = confusion_matrix(y_test, y_pred) print('Confusion Matrix:') print(cm)
```

## 4. Feature Scaling + User Prediction

StandardScaler is applied AFTER training the initial model (for the user-input prediction step). Note: Ideally scaling should happen before training.

```
# StandardScaler: transforms features to mean=0, std=1
sc = StandardScaler() X_train = sc.fit_transform(X_train) # Fit on train, then transform X_test = sc.transform(X_test) # Only transform test (no fitting)
# Get user input — age and salary
age = float(input('Enter age: ')) salary = float(input('Enter salary: '))
# Scale user input the same way training data was scaled
scaled_input = sc.transform([[age, salary]]) prediction = classifier.predict(scaled_input) print(f'The predicted class for age {age} and salary {salary} is: {prediction[0]}')
```

***Note:*** *The model was trained on unscaled data, but prediction uses scaled input — this is inconsistent. Best practice: scale data BEFORE fitting the model.*

# Practical 7 – Text Analytics

Uses NLTK and sklearn to process natural language: tokenize, remove stopwords, stem, lemmatize, POS tag, and classify sentiment.

---

## 1. Sentence & Word Tokenization

```
# sent_tokenize splits paragraph into individual sentences
import nltk from nltk.tokenize import sent_tokenize nltk.download("punkt") text = """Hello Mr. Smith, how are you doing today? The weather is great..."""  tokenized_text = sent_tokenize(text) print(tokenized_text)
# word_tokenize splits text into individual words and punctuation
from nltk.tokenize import word_tokenize tokenized_word = word_tokenize(text)
```

## 2. Frequency Distribution

```
# FreqDist counts how many times each word appears
from nltk.probability import FreqDist fdist = FreqDist(tokenized_word) print(fdist)
```

## 3. Stopword Removal

Stopwords are common words (the, is, in, at) that carry no meaningful information for analysis.

```
# Load English stopwords list ('the', 'is', 'are', 'in', etc.)
nltk.download("stopwords") from nltk.corpus import stopwords stop_words = set(stopwords.words("english")) # Keep only words NOT in stopwords list filtered_sent = [w for w in tokenized_word if w not in stop_words]
```

## 4. Stemming

Stemming cuts word suffixes to find the root: 'running' -> 'run', 'flies' -> 'fli'. Fast but may produce non-real words.

```
# PorterStemmer is the most common English stemming algorithm
from nltk.stem import PorterStemmer ps = PorterStemmer() stemmed_words = [ps.stem(w) for w in filtered_sent]
```

## 5. Lemmatization

Lemmatization uses vocabulary and morphology to return the base dictionary form: 'flying' -> 'fly', 'better' -> 'good'. More accurate than stemming.

```
# 'v' = verb; lemmatize knows 'flying' is a verb form of 'fly'
from nltk.stem.wordnet import WordNetLemmatizer nltk.download("wordnet") lem = WordNetLemmatizer() word = "flying" print("Lemmatized:", lem.lemmatize(word, "v")) # fly print("Stemmed:", ps.stem(word)) # fli
```

## 6. POS Tagging

Part-of-Speech tagging labels each word: NN (noun), VBD (past verb), IN (preposition), NNP (proper noun), etc.

```
# pos_tag assigns grammatical roles to each token
sent = "Albert Einstein was born in Ulm, Germany in 1879." tokens = nltk.word_tokenize(sent) nltk.download("averaged_perceptron_tagger") nltk.pos_tag(tokens) # [("Albert","NNP"), ("Einstein","NNP"), ("was","VBD"), ...]
```

## 7. Sentiment Analysis with CountVectorizer

```
# Read movie/phrase sentiment dataset (0-4 scale)
data = pd.read_csv('train.tsv', sep='\t') data.Sentiment.value_counts() # Count phrases per sentiment level
# CountVectorizer converts text to token count matrix (Bag-of-Words)
from sklearn.feature_extraction.text import CountVectorizer from nltk.tokenize import RegexpTokenizer token = RegexpTokenizer(r'[a-zA-Z0-9]+') # Only alphanumeric tokens cv = CountVectorizer(lowercase=True, stop_words='english', ngram_range=(1,1), tokenizer=token.tokenize) text_counts = cv.fit_transform(data['Phrase'])
# Train MultinomialNB on word counts — good for text classification
from sklearn.naive_bayes import MultinomialNB clf = MultinomialNB().fit(X_train, y_train) predicted = clf.predict(X_test) print('Accuracy:', metrics.accuracy_score(y_test, predicted))
```

## 8. TF-IDF Vectorization

TF-IDF (Term Frequency-Inverse Document Frequency) gives higher weight to rare but important words and downweights common words like 'the'.

```
# TfidfVectorizer is often better than simple CountVectorizer for NLP
from sklearn.feature_extraction.text import TfidfVectorizer tf = TfidfVectorizer() text_tf = tf.fit_transform(data['Phrase'])
# Retrain same MultinomialNB model with TF-IDF features
clf = MultinomialNB().fit(X_train, y_train) predicted = clf.predict(X_test) print('MultinomialNB Accuracy:', metrics.accuracy_score(y_test, predicted))
```

# Practical 8 – Data Visualization 1

Dataset: train.csv (Titanic) | Goal: Explore various seaborn plot types for univariate and bivariate analysis.

---

## 1. Dataset Overview

```
# Load Titanic dataset — 891 passengers, 12 columns
dataset = pd.read_csv('train.csv') dataset.info() # Column types and non-null counts dataset.describe() # Summary statistics for numeric columns dataset.dtypes # Data type of each column dataset.isnull().sum() # Count of missing values per column
```

## 2. Distribution Plot (distplot)

```
# distplot = histogram + KDE (Kernel Density Estimate) curve
sns.distplot(dataset['Fare']) # With KDE sns.distplot(dataset['Fare'], kde=False) # Histogram only sns.distplot(dataset['Fare'], kde=False, bins=10) # 10 bins
```

## 3. Joint & Pair Plots

```
# jointplot: scatter + marginal histograms for 2 variables
sns.jointplot(x='Age', y='Fare', data=dataset)
# pairplot: scatter matrix for ALL numeric column pairs
sns.pairplot(dataset)
```

## 4. Rug Plot

```
# rugplot: small vertical ticks showing individual data points
sns.rugplot(dataset['Fare'])
```

## 5. Bar Plot

```
# barplot: mean Fare by Sex; error bars show confidence interval
sns.barplot(x='Sex', y='Fare', data=dataset)
# estimator=np.std shows STANDARD DEVIATION instead of mean
sns.barplot(x='Sex', y='Age', data=dataset, estimator=np.std)
```

## 6. Count, Box & Violin Plots

```
# countplot: count of occurrences for each Fare value
sns.countplot(x='Fare', data=dataset)
# boxplot: IQR box + whiskers + outlier dots
sns.boxplot(x='Sex', y='Age', data=dataset) sns.boxplot(x='Survived', y='Fare', data=dataset)
# hue='Survived' adds color split by survival within each gender
sns.boxplot(x='Sex', y='Age', data=dataset, hue='Survived')
# violinplot: combines boxplot + KDE — shows full distribution shape
sns.violinplot(x='Sex', y='Age', data=dataset, hue='Survived')
# stripplot: individual data points overlaid by category
sns.stripplot(x='Sex', y='Age', data=dataset)
# histplot: 2D histogram — bivariate frequency distribution
sns.histplot(data=dataset, x='Fare', y='Survived')
```

# Practical 9 – Data Visualization 2

Datasets: Titanic (CSV) and Tips (built-in seaborn) | Goal: Deeper comparison using boxplots with hue.

---

## 1. Load Both Datasets

```
# Titanic from CSV, tips from seaborn's built-in datasets
data = pd.read_csv('/content/train.csv') from seaborn import load_dataset tips =
load_dataset('tips') # Restaurant tipping data
```

## 2. Boxplots – Titanic

```
# Age distribution by gender — shows median, IQR, whiskers, outliers
sns.boxplot(x='Sex', y='Age', data=dataset)
# Add hue to split each gender box by survival (0=died, 1=survived)
sns.boxplot(x='Sex', y='Age', data=dataset, hue='Survived')
# Fare paid by survival status — higher fare survivors lived longer?
sns.boxplot(x='Survived', y='Fare', data=dataset)
```

***Note:*** *Repeated boxplots in this file appear to be for practice/comparison. In an exam, focus on interpreting: median line position, IQR width, whisker length, and outlier dots beyond whiskers.*

# Practical 10 – Data Visualization 3

Dataset: data10.csv (likely IRIS) with generic column names col1-col5 | Goal: Histograms and combined boxplots.

---

## 1. Load & Rename Columns

```
# Load CSV and rename columns generically (col1..col5)
df = pd.read_csv("data10.csv") df.columns = ["col1", "col2", "col3", "col4", "col5"]
# Count number of columns
column = len(list(df)) column
```

## 2. Feature Types & Summary

```
# df.info() tells us dtype and non-null count — identifies numeric vs object
df.info() # Expected: 4 numeric (float), 1 object (species name)
# Get unique values of the categorical column
np.unique(df["col5"])
# Statistical summary: count, mean, std, min, quartiles, max
df.describe()
```

## 3. Histograms for All Features

Histograms show the frequency distribution of each feature — useful to detect skewness, bimodal distributions, and outliers.

```
# Create 2x2 grid of subplots — one histogram per numeric column
fig, axes = plt.subplots(2, 2, figsize=(16, 8)) axes[0,0].set_title("Distribution of First Column") axes[0,0].hist(df["col1"]) # Histogram of col1 axes[0,1].set_title("Distribution of Second Column") axes[0,1].hist(df["col2"]) # Histogram of col2 axes[1,0].set_title("Distribution of Third Column") axes[1,0].hist(df["col3"]) # Histogram of col3 axes[1,1].set_title("Distribution of Fourth Column") axes[1,1].hist(df["col4"]) # Histogram of col4
```

## 4. Combined Boxplot for All Features

A combined boxplot puts all four numeric columns side by side to compare their distributions and identify outliers.

```
# Put all 4 columns in a list for the combined boxplot
data_to_plot = [df["col1"], df["col2"], df["col3"], df["col4"]]
# Create figure and axes, then draw boxplot
sns.set_style("whitegrid") fig = plt.figure(1, figsize=(12, 8)) ax = fig.add_subplot(111) # Single subplot (1 row, 1 col, position 1) bp = ax.boxplot(data_to_plot)
# seaborn boxplot for column pairs (grouped comparison)
sns.boxplot(x='col1', y='col2', data=df) sns.boxplot(x='col3', y='col4', data=df)
```

***Note:*** *For box 2: if IQR ~ 0.75, then values beyond Q3 + 1.5*IQR (approx 4.05) are flagged as outliers. Each box's whiskers extend to 1.5*IQR beyond Q1 and Q3.*

---

## Quick Reference: All Key Concepts

| Topic | Key Concept | sklearn/lib Function |
|---|---|---|

| | | |
|---|---|---|
| Normalization | Scale to [0,1] | MinMaxScaler.fit_transform() |
| Label Encoding | String categories to integers | LabelEncoder.fit_transform() |
| One-Hot Encoding | Binary columns per category | OneHotEncoder / pd.get_dummies() |
| Missing Values | Fill with stat measures | df.fillna(mean/median/std) |
| Linear Regression | Predict continuous values | LinearRegression / np.polyfit() |
| Logistic Regression | Binary classification | LogisticRegression.predict() |
| Naive Bayes | Probabilistic classification | GaussianNB / MultinomialNB |
| Text Vectorization | Text to numeric features | CountVectorizer / TfidfVectorizer |
| NLP Preprocessing | Clean text for analysis | NLTK: tokenize/stem/lemmatize |
| R2 Score | Goodness of fit (0 to 1) | r2_score(y_true, y_pred) |
| Confusion Matrix | TP/TN/FP/FN classification grid | confusion_matrix().ravel() |
| Seaborn Plots | Statistical visualizations | boxplot/violinplot/distplot |